

URL Shortener – Proof of Concept (PoC)

Name: Rudra Sawant

Intern ID: 211

Organization: Digisuraksha Parhari Foundation

Project: URL Shortener using Node.js + SQLite + nanoid

Objective

To build a simple web-based URL Shortener that accepts long URLs and returns unique, short, user-friendly versions with redirection and basic click tracking.

Problem Statement (Simplified)

Long URLs are inconvenient to share or remember, especially on messaging and social media platforms. This project solves the problem by offering:

A field to input a long URL

A unique short version as output

Redirection functionality

Basic tracking of URL usage

Tools Used

Tool	Purpose
------	---------

Node.js	Backend runtime
Express.js	Web server framework
SQLite3	Lightweight local database
nanoid	Generates secure, unique short codes
EJS	Template engine for dynamic views
HTML/CSS	Frontend layout

Project Structure

```
url-shortener/  
├── public/  
│   └── index.html  
├── views/  
│   └── result.ejs  
├── server.js  
├── package.json  
└── urls.db
```

Use Case Example

A user wants to share a messy long link on social media.

They paste the link in the tool → get a neat short link → share it.

Anyone clicking it gets redirected to the original page, and their visit is logged.

Frontend Form (HTML)

```
<form action="/shorten" method="POST">  
  <input type="text" name="longUrl" placeholder="Enter long URL" required />  
  <button type="submit">Shorten</button>  
</form>
```

Backend Setup

```
npm init -y  
npm install express body-parser sqlite3 ejs nanoid
```

This installs:

express – web server

body-parser – form input handler

sqlite3 – local database

ejs – for displaying results

nanoid – to generate unique IDs

Database Setup

```
const sqlite3 = require('sqlite3').verbose();
const db = new sqlite3.Database('./urls.db');

db.serialize(() => {
  db.run('CREATE TABLE IF NOT EXISTS urls (id TEXT PRIMARY KEY, longUrl TEXT)');
  db.run('CREATE TABLE IF NOT EXISTS clicks (id INTEGER PRIMARY KEY
AUTOINCREMENT, shortId TEXT, timestamp TEXT, ip TEXT)');
});
```

Short Code Logic

```
const { nanoid } = require('nanoid');
const id = nanoid(7);
```

Shorten Route

```
app.post('/shorten', (req, res) => {
  const longUrl = req.body.longUrl;
  const id = nanoid(7);

  db.run('INSERT INTO urls (id, longUrl) VALUES (?, ?)', [id, longUrl], (err) => {
    if (err) return res.send('DB Error');
    res.render('result', { shortUrl: `http://localhost:${PORT}/${id}` });
  });
});
```

Result Page (EJS View)

```
<h2>Your Short URL</h2>
<a href="<%= shortUrl %>"><%= shortUrl %></a>
```

Redirection Logic

```
app.get('/:id', (req, res) => {
  const id = req.params.id;

  db.get('SELECT longUrl FROM urls WHERE id = ?', [id], (err, row) => {
    if (err || !row) return res.send('URL not found!');

    const timestamp = new Date().toISOString();
    const ip = req.ip;

    db.run('INSERT INTO clicks (shortId, timestamp, ip) VALUES (?, ?, ?)', [id, timestamp, ip]);
    res.redirect(row.longUrl);
  });
});
```

js

```
app.post('/shorten', (req, res) => {
  const longUrl = req.body.longUrl;
  const id = nanoid(7);

  db.run('INSERT INTO urls (id, longUrl) VALUES (?, ?)', [id, longUrl], (err) => {
    if (err) return res.send('DB Error');
    res.render('result', { shortUrl: `http://localhost:${PORT}/${id}` });
  });
});
```



User Flow

1. Open tool (e.g. <http://localhost:3000>)
2. Enter long URL
3. Click "Shorten"

4. Get short URL (e.g., <http://localhost:3000/a1b2c3d>)
5. Share it
6. Redirection & tracking happen automatically when clicked

Test Cases

Scenario	Expected Result
Valid long URL	Short URL generated
Invalid URL	Error message shown
Click on short URL	Redirects to original link
Invalid short code	"URL not found" message

Benefits

- Makes link sharing clean and easy
- Uses safe, random short codes
- Stores all data locally (no external DB needed)
- Educational for understanding routing, forms, databases
- Simple tech stack; easy to maintain and extend

Who Can Use This Tool?

- General Users: Clean link sharing
- Developers/Students: Learn routing, DB, redirection
- Marketers: Track campaign link usage
- Organizations: Brand internal links
- Cybersecurity Interns: Understand redirection techniques (and possible abuse)

Conclusion

This project shows how a basic full-stack app works by:

Taking a long URL

Creating a short version using nanoid

Redirecting users on short URL access

Logging click events with timestamp and IP

It's a great beginner-friendly project for web development and cybersecurity awareness.